## Run-Length Encoding

Data files frequently contain the same character repeated many times in a row. For example, text files use multiple spaces to separate sentences, indent paragraphs, format tables & charts, etc. Digitized *signals* can also have runs of the same value, indicating that the signal is not changing. For instance, an image of the nighttime sky would contain long runs of the character or characters representing the black background. Likewise, digitized music might have a long run of zeros between songs. Run-length encoding is a simple method of compressing these types of files.

Figure 27-1 illustrates run-length encoding for a data sequence having frequent runs of *zeros*. Each time a zero is encountered in the input data, *two* values are written to the output file. The first of these values is a zero, a flag to indicate that run-length compression is beginning. The second value is the number of zeros in the run. If the average run-length is longer than two, compression will take place. On the other hand, many single zeros in the data can make the encoded file larger than the original.

Many different run-length schemes have been developed. For example, the input data can be treated as individual bytes, or groups of bytes that represent something more elaborate, such as floating point numbers. Run-length encoding can be used on only *one* of the characters (as with the *zero* above), *several* of the characters, or *all* of the characters.

A good example of a generalized run-length scheme is **PackBits**, created for Macintosh users. Each byte (eight bits) from the input file is replaced by nine bits in the compressed file. The added ninth bit is interpreted as the *sign* of the number. That is, each character read from the input file is between 0 to 255, while each character written to the encoded file is between -255 and 255. To understand how this is used, consider the input file: 1,2,3,4,2,2,2,2,4, and the compressed file generated by the PackBits algorithm: 1,2,3,4,2,-3,4. The compression program simply transfers each number from the input file to the compressed file, with the exception of the run: 2,2,2,2. This is represented in the compressed file by the two numbers: 2,-3. The first number ("2") indicates what character the run consists of. The second number ("-3") indicates the number of characters in the run, found by taking the absolute value and adding one. For instance, 4,-2 means 4,4,4; 21,-4 means 21,21,21,21,21, etc.

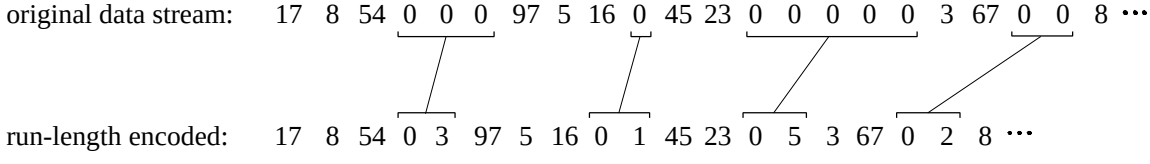


FIGURE 27-1
Example of run-length encoding. Each run of zeros is replaced by two characters in the compressed file: a zero to indicate that compression is occurring, followed by the number of zeros in the run.